

University of Brescia, Italy
Machine Learning & Data Mining

Evaluating Hypotheses and Classification Algorithms

Alfonso E. Gerevini

Univ. of Brescia, Italy

Evaluating Hypotheses and Classification Algorithms

- Estimating general accuracy of hypotheses from data
- Sample error, true error
- Confidence intervals for observed hypothesis error
- Estimators
- Comparing learning methods
- Other performance metrics in classification

Statistical methods for estimating accuracy

Three important questions:

- Given accuracy of h over a limited sample of data, how well does this estimate its accuracy over additional examples?
- Given that h outperforms h' over some sample of data, how probable is it that h is more accurate in general?
- When data is limited what is the best way to use data to both learn h and estimate its accuracy?

Example

Consider a typical classification problem:

$$f : X \rightarrow Y$$

$\mathcal{D}$: *probability distribution over X*

S : *sample of n instances drawn from X (according to distribution $\mathcal{D}$) and for which we know $f(x)$*

E.g., X = all persons; f = persons intending to buy new skies this year

Consider a hypothesis h , solution of a learning algorithm:

- What is the best estimate of the accuracy of h over **future instances** drawn from the same distribution?
- What is the probable error in this accuracy estimate?

Two Definitions of Error

The **true error** of hypothesis h with respect to target function f and distribution $\mathcal{D}$ is the probability that h will misclassify an instance drawn at random according to $\mathcal{D}$.

$$\text{error}_{\mathcal{D}}(h) \equiv \Pr_{x \in \mathcal{D}}[f(x) \neq h(x)]$$

The **sample error** of h with respect to target function f and data sample S is the portion of examples that h misclassifies

$$\text{error}_S(h) \equiv \frac{1}{n} \sum_{x \in S} \delta(f(x) \neq h(x))$$

where $\delta(f(x) \neq h(x))$ is 1 if $f(x) \neq h(x)$, and 0 otherwise.

Note: $\text{accuracy}_S(h) \equiv 1 - \text{error}_S(h)$

Sample Error as Estimator of True Error

We wish to know the true error $error_{\mathcal{D}}(h)$

⇒ But the **true error** cannot be computed...

Then, how well does $error_S(h)$ estimate $error_{\mathcal{D}}(h)$?

⇒ The **sample error** is computed only on a small data sample...

- $error_S(h)$ is an **estimator** of $error_{\mathcal{D}}(h)$

Estimators as random variables

Experiment:

- 1 Partition the data set D ($D = T \cup S$, $T \cap S = \emptyset$, $|T| = 2/3|D|$)
- 2 Compute a hypothesis h using training set T
- 3 Measure $error_S(h)$

$error_S(h)$ is a **random variable**: result of an experiment for a particular S

$error_S(h)$ is an **unbiased estimator** for $error_D(h)$ [see next slides]

Given observed $error_S(h)$ what can we conclude about $error_D(h)$?

Problems in Estimating the True Error

Estimation bias (should not be confused with inductive bias!)

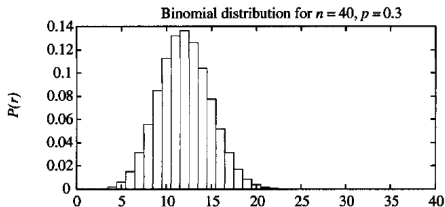
$$\text{bias} \equiv E_{S \subset X}[\text{error}_S(h)] - \text{error}_{\mathcal{D}}(h)$$

$E_{S \subset X}[\cdot]$ = mean of $\cdot$ over subsets of X , $|S| = n$

- If S is the same training set used to compute h , $\text{error}_S(h)$ is **optimistically** biased ($\text{error}_S(h) - \text{error}_{\mathcal{D}}(h)$ can be < 0)
- For unbiased estimation, h and S must be chosen **independently**
- Even with unbiased estimation, $\text{error}_S(h)$ for a **particular** S may *BE DIFFERENT* from $\text{error}_{\mathcal{D}}(h)$.

Why $error_S(h)$ is unbiased?

- Observing r errors in $n = |S|$ examples same as r heads in n coin flip experiments
- The Binomial distribution characterises random variable $X = r$
- $P(X = r) : \frac{n!}{r!(n-r)!} p^r (1-p)^{n-r}$; $E[X] = np$; $Var(X) = np(1-p)$
- p = probability of heads on a single coin toss = probability of misclassification of a single instance = $error_D(h)$



- $E[error_S(h)] = E[\frac{r}{n}] = p$, because $E[r] = np$ and n is a constant
 $\implies E_{S \subset X}[error_S(h)] - error_D(h) = 0$

Confidence Intervals

An $N\%$ **confidence interval** for some parameter p is an interval that is expected with probability $N\%$ ($N/100$) to contain p .

If

- S contains n examples, drawn independently of h and each other
- $n \geq 30$

Then statistical theory says:

- With *approximately* 95% probability, $error_D(h)$ lies in interval

$$error_S(h) \pm 1.96 \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}}$$

Confidence Intervals More in General

If

- S contains n examples, drawn independently of h and each other
- $n \geq 30$

Then

- With approximately $N\%$ probability, $error_{\mathcal{D}}(h)$ lies in interval

$$error_S(h) \pm z_N \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}}$$

where

$N\%$:	50%	68%	80%	90%	95%	98%	99%
z_N :	0.67	1.00	1.28	1.64	1.96	2.33	2.58

Assumptions for the confidence intervals

- The hypothesis is discrete-valued
- S drawn at random using the same distribution from which future data will be drawn
- S independent from the training data used for generating h

Approximation is good when $n \geq 30$ and $error_S(h)$ not too close to 0 or 1:

Practical Rule

IF $n \cdot error_S(h)(1 - error_S(h)) \geq 5$ THEN approximation works

Standard deviation and confidence intervals

Standard deviation of $error_S(h)$ is

$$\sigma_{error_S(h)} = \frac{\sigma^r}{n} = \sqrt{\frac{p(1-p)}{n}}$$

Substituting $r/n = error_S(h)$ for p :

$$\sigma_{error_S(h)} \sim \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}}$$



$$error_S(h) \pm z_N \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}} \text{ approximates } error_S(h) \pm z_N \sigma_{error_S(h)}$$

Comparing two hypotheses

Given two hypotheses h_1, h_2 , the true comparison is

$$d \equiv \text{error}_{\mathcal{D}}(h_1) - \text{error}_{\mathcal{D}}(h_2)$$

and its estimator is

$$\hat{d} \equiv \text{error}_{S_1}(h_1) - \text{error}_{S_2}(h_2)$$

Instances in S_1 and S_2 chosen with the same probability distribution

It can be proved that $\hat{d}$ is an *unbiased estimator* for d , if and only if h_1, h_2, S_1 and S_2 are independent each other:

$$E[\hat{d}] = d$$

Comparing two hypotheses with Confidence Interval

- For large $n_1 = |S_1|$ and $n_2 = |S_2|$ $error_{S_1}(h_1)$ $error_{S_2}(h_2)$ are approximately normal distributions
- $error_{S_1}(h_1) - error_{S_2}(h_2)$ is an approximately normal distribution with variance the sum of the relative variances

⇒ Approximate N% confidence interval for d is

$$\hat{d} \pm z_N \sqrt{\frac{error_{S_1}(h_1)(1 - error_{S_1}(h_1))}{n_1} + \frac{error_{S_2}(h_2)(1 - error_{S_2}(h_2))}{n_2}}$$

Comparing two hypotheses

If there is scarcity of training example, we can use $S_1 = S_2$ (same test set for h_1 and h_2)

If $S_1 = S_2 = S$, then the estimator

$$\hat{d} \equiv error_S(h_1) - error_S(h_2)$$

has usually a smaller variance (std deviation) and the confidence interval is still correct but overly conservative (tighter).

Evaluation of a learning algorithm

How can we evaluate the performance of a learning algorithm?

h is the solution of learning algorithm L when using a training set T
 $h = L(T)$

$error_S(h)$ is the result of only one experiment and the confidence interval can be large.

We can perform many experiments and compute $error_{S_i}(h)$ for different **independent** sample data S_i .

⇒ **K-Fold Cross Validation** method

K-Fold Cross Validation

- ① Partition data set D into k disjoint sets $S_1, S_2, \dots, S_k$ ($|S_i| > 30$)
- ② For $i = 1, \dots, k$ do
 - use S_i as test set, and the remaining data as training set T_i
 - $T_i \leftarrow \{D - S_i\}$
 - $h_i \leftarrow L(T_i)$ where $L(\cdot)$ output hypothesis of learner L built from T_i
 - $\delta_i \leftarrow \text{error}_{S_i}(h_i)$
- ③ Return

$$\text{error}_{L,D} \equiv \frac{1}{k} \sum_{i=1}^k \delta_i$$

Note: $\text{accuracy}_{L,D} = 1 - \text{error}_{L,D}$

Comparing learning algorithms L_A and L_B

Which algorithm is better?

We would like to estimate:

$$E_{S \subset \mathcal{D}}[\text{error}_{\mathcal{D}}(L_A(S)) - \text{error}_{\mathcal{D}}(L_B(S))]$$

where $L(S)$ is the hypothesis output by learner L using training set S

i.e., the expected difference in true error between hypotheses output by learners L_A and L_B , when trained using randomly selected training set S of size n drawn according to distribution $\mathcal{D}$.

This measure can be again approximated by a K-Fold Cross Validation.

Comparing learning algorithms L_A and L_B

Use K-Fold Cross Validation to compare algorithms L_A and L_B .

- ① Partition data set $D \subset \mathcal{D}$ into k disjoint sets $S_1, S_2, \dots, S_k$ ($|S_i| > 30$)
- ② For i from 1 to k , do
 - use S_i as test set, and the remaining data as training set T_i*
 - $T_i \leftarrow \{D - S_i\}$
 - $h_A \leftarrow L_A(T_i)$
 - $h_B \leftarrow L_B(T_i)$
 - $\delta_i \leftarrow \text{error}_{S_i}(h_A) - \text{error}_{S_i}(h_B)$
- ③ Return

$$\bar{\delta} \equiv \frac{1}{k} \sum_{i=1}^k \delta_i$$

Note: if $\bar{\delta} < 0$ we can **estimate** that L_A is better than L_B .

Comparing learning algorithms L_A and L_B

The training sets in this algorithm are **not** independent (they overlap!)

More correct to view the algorithm as producing an estimate of

$$E_{S \subset D}[\text{error}_{\mathcal{D}}(L_A(S)) - \text{error}_{\mathcal{D}}(L_B(S))]$$

instead of

$$E_{S \subset D}[\text{error}_{\mathcal{D}}(L_A(S)) - \text{error}_{\mathcal{D}}(L_B(S))]$$

(S over D instead of $\mathcal{D}$!)

but even this approximation is better than no comparison...

Some Methods of Evaluation

- **Holdout:** Reserve $2/3$ for training and $1/3$ for testing
- **Random subsampling:** Repeated holdout
- **Cross validation:** Partition data into k disjoint subsets and
 - *k-fold:* train on $k-1$ partitions, test on the remaining one
 - *Leave-one-out:* $k=n$
- **Bootstrap:** Sampling with replacement

Bootstrap

- When we randomly choose an example for training, we can re-choose examples already selected
- If we choose n times an example from a dataset of n distinct elements, on average we obtain 63.2% of the n examples
- These examples are called *bootstrap samples* (used the training; the rest for testing)
- We repeat b times and we use a weighted average of the performance on the test sets:

$$Accuracy_{boot} = \frac{1}{b} \sum_{i=1}^b (0.632 \times \epsilon_i + 0.368 \times acc_s)$$

ϵ_i = accuracy of the model with bootstrap training set i

acc_s = accuracy of the model with all examples (test set = training set)

Other performance metrics in classification

	Predicted class	
True Class	Yes	No
Yes	TP: True Positive	FN: False Negative
No	FP: False Positive	TN: True Negative

Error rate = $| \text{errors} | / | \text{instances} | = (FN + FP) / N$

Accuracy = $1 - \text{Error rate} = (TP + TN) / N$

Recall/Sensitivity = $| \text{true positives} | / | \text{real positives} | = TP / (TP + FN)$

Specificity = $| \text{true negative} | / | \text{real negatives} | = TN / (TN + FP)$

Precision = $| \text{true positives} | / | \text{predicted positives} | = TP / (TP + FP)$

False alarm rate = $| \text{false positives} | / | \text{real negatives} | = FP / (FP + TN)$
 $= 1 - \text{Specificity}$

F1 Score: media armonica tra precision e recall, serve per misurare l'accuratezza della predizione

Media armonica p e r:

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

$$\frac{2}{\frac{1}{p} + \frac{1}{r}} = \frac{2 \cdot p \cdot r}{p + r}$$
$$F1 = \frac{2TP}{2TP + FN + FP} \leftarrow FN \text{ e } FP \text{ stessa importanza}$$

F2 Score: media armonica pesata tra precision e recall, serve per misurare l'accuratezza della predizione dando maggiore enfasi ai falsi negativi

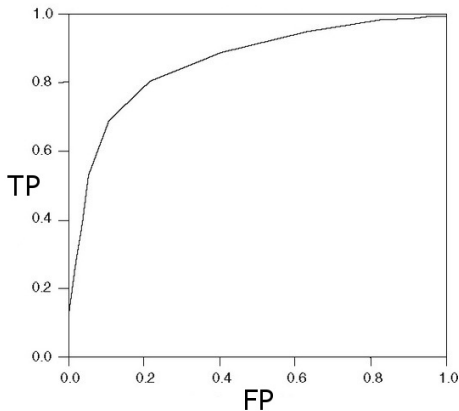
$$F_{\beta} = (1 + \beta^2) \frac{p \cdot r}{\beta^2 \cdot p + r}$$
$$F2 = \frac{5 \cdot \text{precision} \cdot \text{recall}}{(4 \cdot \text{precision} + \text{recall})}$$
$$F2 = \frac{5 \cdot TP}{5 \cdot TP + 4 \cdot FN + FP} \leftarrow FN \text{ ha } > \text{ importanza di } FP$$

media armonica β -pesata

Se ci sono più classi da predire, si usa la **Macro F2 Score** che è la media aritmetica delle F2 score di tutte le classi.

ROC (Receiver Operator Curve)

ROC (Receiver Operator Curve) curve: plot of $y = \text{True Positive rate (Sensitivity)}$ on $x = \text{False Positive rate (1 - Specificity)}$ when we vary some parameters of the algorithm (for instance, the decision threshold)



Confusion Matrix

In a classification problem with many classes, we can compute how many times an instance of class C_i is classified in class C_j .

	C_1	C_2	C_3	C_4	C_5
C_1					
C_2					
C_3					
C_4					
C_5					

Main diagonal contains accuracy for each class.

Outside the diagonal, the errors. It is possible to see which classes are more often confused.